

RuFLEX: draft article text

Название

RuFLEX: гибридная платформа глубокого нечеткого обучения для геоданных с интерпретируемым анализом правил, факторов и скрытых концептов

Аннотация

В работе представлена платформа RuFLEX для гибридного глубокого нечеткого обучения, ориентированная на обработку геоданных в задачах, где высокая прогностическая точность должна сочетаться с интерпретируемостью, экспертной верифицируемостью и прозрачностью логики вывода. В отличие от традиционных непрозрачных подходов предложенная архитектура объединяет дифференцируемые нечеткие слои, модули глубокого нечеткого формирования признаков, расширяемый пространственный контур глубоких нечетких сверток и механизмы принятия решений на основе правил в едином дифференцируемом вычислительном контуре, реализованном в экосистеме PyTorch. Модельный контур поддерживает совместное обучение параметров функций принадлежности, базы правил и глубоких представлений признаков при сохранении семантически интерпретируемого вывода на уровне факторов и правил. Платформа включает программное ядро в виде программного интерфейса Python, веб-интерфейс визуального проектирования моделей и панель интерпретации, обеспечивающую анализ функций принадлежности, структуры правил, скрытых нечетких концептов и вклада факторов в итоговое решение. Перспективным направлением развития является расширение контура объяснимости в сторону логической прослеживаемости правил и контрфактуального анализа.

1. Введение

В задачах геоаналитики, пространственной классификации, прогнозирования и интеллектуальной поддержки принятия решений высокая точность предсказания не может рассматриваться отдельно от интерпретируемости модели. Для прикладных постановок важны не только итоговые значения метрик, но и прозрачность логики вывода, возможность экспертной проверки базы правил, а также анализ того, какие факторы определили результат на конкретном объекте. По этой причине непрозрачные архитектуры, хотя и способны давать высокое качество прогноза, не снимают задачу полностью.

Нечеткие и нейро-нечеткие методы, напротив, сохраняют связь между обучаемым представлением признаков и семантически понятным выводом, поэтому продолжают занимать важное место в исследованиях такого типа [1].

Вместе с тем современная постановка задачи требует уже не отдельной модели, а целостной программной среды. Если исследователь хочет сравнить плоскую интерпретируемую конфигурацию и более глубокий контур формирования признаков, ему недостаточно иметь только вычислительное ядро. Необходимы средства задания переменных, термов, функций принадлежности и правил, средства обучения, визуального проектирования, воспроизводимой постановки эксперимента и содержательной интерпретации результатов. Именно эта логика определяет платформенный характер RuFLEX. В работе она рассматривается не как единичный вариант нейро-нечеткой сети, а как расширяемая среда для гибридного глубокого нечеткого обучения.

Цель статьи состоит в описании архитектуры платформы RuFLEX, формализации ее модельного контура и прикладной демонстрации работы на задачах обработки геоданных. Основной акцент делается на модульной структуре платформы, разделении вычислительного и платформенного контуров, совместном обучении параметров функций принадлежности, базы правил и внутренних представлений, а также на средствах интерпретации, которые позволяют анализировать поведение модели на уровне признаков, правил и скрытых нечетких концептов.

2. Архитектура и модульный состав платформы

RuFLEX реализована как платформенный слой поверх вычислительного ядра `ruanfis`, происходящего из репозитория `deer-neuro-fuzzy` [7]. Такое построение выбрано сознательно. Вычислительное ядро отвечает за дифференцируемые нечеткие преобразования, вычисление активаций правил, локальные правилые блоки, базовые процедуры обучения и исполнение модельного контура. Платформенный слой развивает собственную объектную модель проекта, сериализацию, интерфейсную оболочку, средства визуализации, управление вычислительными опытами и контур интерпретации. За счет этого сохраняется преемственность глубокого нечеткого ядра, но сама платформа получает возможность развиваться как самостоятельная исследовательская и прикладная среда.

Архитектура платформы строится как совокупность двух взаимосвязанных контуров. Первый контур является вычислительным. В нем находятся

фаззификация, вычисление активаций правил, построение скрытых концептов и формирование решающего слоя. Второй контур является платформенным. Он охватывает описание данных, объект проекта, средства настройки моделей, журналирование обучения, визуальное сопровождение экспериментов, анализ результатов и экспорт материалов. Такое разделение принципиально важно для статьи, поскольку RuFLEX заявляется именно как платформа, а не как единичная реализация одной модели.

На семантическом уровне проекта задаются входные и выходные переменные, диапазоны их значений, роли признаков, режимы нормализации и принадлежность переменной к входному, скрытому или выходному уровню. Здесь же формируются термы и функции принадлежности. Для каждой переменной могут быть заданы гауссовы, колоколообразные, треугольные и трапециевидные функции принадлежности, причем для дифференцируемых режимов основную роль играют гладкие семейства, пригодные для совместной оптимизации. Тем самым платформа связывает предметное описание данных с обучаемым нечетким представлением.

Следующий уровень образует правилковая подсистема. В ней задаются антецеденты, типы агрегации, веса правил, параметры заключений, локальные базы правил и ограниченные блоки, препятствующие взрывному росту числа правил. Локальная блочная организация является принципиальной особенностью платформы, поскольку именно она делает возможным переход от плоского интерпретируемого режима к более глубокому построению признаков без полного разрушения прозрачности вывода. На этой основе формируется модельный слой, который включает плоскую интерпретируемую конфигурацию, режим глубокого нечеткого формирования признаков и расширяемый пространственный каркас.

Подсистема обучения объединяет начальную инициализацию, поэтапное обучение, циклы уточнения и совместную оптимизацию параметров функций принадлежности, правил и внутренних представлений. Она дополняется журналированием, сохранением лучших состояний, историей обучения и воспроизводимым запуском вычислительных опытов.

Подсистема интерпретации обеспечивает визуализацию функций принадлежности, анализ структуры правил, активаций и нормированных весов, анализ вклада правил в скрытые концепты и вклада скрытых концептов в итоговый ответ. На уровне пользовательского контура эти средства переходят в программный интерфейс Python, веб-интерфейс

визуального проектирования, панель интерпретации и экспорт артефактов статьи. Схема 1 фиксирует разделение вычислительного ядра и платформенных сервисов, схема 2 показывает модельный контур с двумя основными режимами, а схема 3 описывает путь от выбранного объекта к аналитическим материалам.

3. Математическая структура модельного контура

Пусть входной объект описывается вектором признаков $x = (x_1, \dots, x_d)$, а нормализованное представление входа задается вектором $z^{(0)}(x) = (z_1^{(0)}, \dots, z_d^{(0)})$, где $z_i^{(0)} = \varphi_i(x_i)$ и φ_i обозначает отображение нормализации. Для i -й переменной и j -го терма задается функция принадлежности $\mu_{ij}(z_i^{(0)}; \theta_{ij})$, где θ_{ij} обозначает набор настраиваемых параметров. В частности, для гауссовой функции принадлежности используется выражение

$$\mu_{ij}(z_i^{(0)}) = \exp(- (z_i^{(0)} - c_{ij})^2 / (2\sigma_{ij}^2)),$$

а для колоколообразной функции принадлежности

$$\mu_{ij}(z_i^{(0)}) = 1 / (1 + |(z_i^{(0)} - c_{ij}) / a_{ij}|^{2b_{ij}}).$$

После фаззификации формируется набор локальных степеней принадлежности, на основе которого вычисляются активации правил. Для r -го правила на уровне l его ненормированная активация вычисляется как

$$w_r^{(l)}(x) = \prod_{m=1}^{k_r} \mu_{i_m j_m}^{(l)}(z_{i_m}^{(l-1)}(x); \theta_{i_m j_m}^{(l)}),$$

где $z^{(l-1)}(x)$ обозначает представление предыдущего уровня, а k_r есть длина правила. После этого вводится нормированный вес правила

$$\bar{w}_r^{(l)}(x) = w_r^{(l)}(x) / (\sum_s w_s^{(l)}(x) + \varepsilon),$$

где $\varepsilon > 0$ предотвращает вырождение знаменателя.

Если речь идет о плоской интерпретируемой конфигурации, то нормированные веса правил непосредственно участвуют в вычислении отклика решающего слоя. В более общем случае, который соответствует глубокому нечеткому формированию признаков, входное пространство разбивается на локальные блоки $B_q^{(l)}$, каждый из которых формирует новый скрытый нечеткий концепт. Для q -го концепта на уровне l используется выражение

$$h_q^{(l)}(x) = \sum_{r \in B_q^{(l)}} \bar{w}_r^{(l)}(x) \cdot g_r^{(l)}(z^{(l-1)}(x)),$$

где $B_q^{(l)}$ есть множество правил блока, формирующего q -й концепт, а $g_r^{(l)}$ задает параметризованное заключение правила. Тогда вектор $h^{(l)}(x)$

образует новое пространство признаков, которое подается на следующий уровень модели. Финальный решающий слой вычисляет ответ

$$\hat{y}(x) = \sum_{r \in B^L} \bar{w}_r^L(x) \cdot g_r^L(h^{L-1}(x)).$$

Такое представление позволяет сохранить интерпретируемость на уровне правил и одновременно наращивать выразительность модели за счет последовательного построения скрытых нечетких концептов. Тем самым скрытый слой рассматривается не как непрозрачный внутренний вектор, а как новое концептуальное пространство, полученное из ограниченного множества локальных правил.

Совместное обучение параметров платформы задается функционалом

$$L = L_{\text{задачи}} + \lambda_1 L_{\text{разреженности}} + \lambda_2 L_{\text{перекрытия}} + \lambda_3 L_{\text{покрытия}} + \lambda_4 L_{\text{концептов}},$$

где $L_{\text{задачи}}$ соответствует основной метрике качества, а остальные слагаемые регулируют компактность базы правил, структуру термов и различимость скрытых концептов. В простейшем виде эти слагаемые могут быть заданы как

$$L_{\text{разреженности}} = \sum_l \sum_r |\alpha_r^l|,$$

$$L_{\text{перекрытия}} = \sum_{(i,j)<k} \int \mu_{ij}(t) \mu_{ik}(t) dt,$$

$$L_{\text{концептов}} = \sum_l \sum_{(q \neq p)} \langle h_q^l, h_p^l \rangle^2.$$

Здесь α_r^l обозначает вес r -го правила, а выражение для $L_{\text{концептов}}$ препятствует вырождению скрытых концептов в близкие по смыслу представления. Благодаря этому обучение ориентировано не только на численное качество, но и на структурную интерпретируемость модели.

Для пространственного контура вводится тензор признаков $X \in \mathbb{R}^{(H \times W \times C)}$. В этом случае нечеткий сверточный отклик m -го ядра на позиции (u, v) может быть записан в виде

$$u_m(u, v) = \sum_{(a,b,c) \in \Omega} \kappa_{(mabc)} \cdot \mu_{(mabc)}(X(u+a, v+b, c)),$$

где Ω задает локальное окно, $\kappa_{(mabc)}$ являются обучаемыми весами, а $\mu_{(mabc)}$ описывают локальные функции принадлежности. В первой версии платформы этот модуль подготовлен как расширяемый каркас, согласованный с основной архитектурой и предназначенный для дальнейшего развития пространственного контура.

Наконец, интерпретация результата строится через вклад правил, концептов и факторов. Для r -го правила в финальном слое локальный вклад может быть

оценен как

$$C_r(x) = \bar{w}_r^{(L)}(x) \cdot g_r^{(L)}(h^{(L-1)}(x)),$$

а вклад q -го скрытого концепта в итоговый ответ может быть задан выражением

$$C_q^{(l)}(x) = \sum_{r \in B_q^{(l)}} C_r^{(l)}(x).$$

После этого вклад исходных признаков восстанавливается через агрегирование по тем блокам и правилам, в которых эти признаки участвовали. В результате интерпретация опирается не на внешний постфактум-анализ, а на внутреннюю структуру самого модельного контура.

4. Экспериментальная постановка

Для прикладной проверки платформы использованы две задачи обработки геоданных на основе набора `sklearn.fetch_california_housing` [3, 4].

Первая задача, California Housing Regression, сохраняет пространственные признаки Latitude и Longitude и использует целевую переменную MedHouseVal. Вторая задача, California Value Binary Geo, строится на том же источнике данных, но в постановке бинарной классификации, где целевая переменная HighValue определяется относительно медианы MedHouseVal. В обеих задачах использовались подвыборки объема 4096 наблюдений, что обеспечило сопоставимость экспериментального контура.

Сравнивались четыре конфигурации платформы: плоская базовая конфигурация, плоская интерпретируемая конфигурация, демонстрационный глубокий режим и расширенный глубокий режим. Такое сравнение позволяет проследить, как изменяется качество при переходе от компактной интерпретируемой конфигурации к более глубокому контуру формирования признаков. Все эксперименты выполнялись в воспроизводимом исследовательском контуре, который автоматически сохраняет метрики, таблицы результатов, графики, панели интерпретации и текстовые отчеты.

5. Результаты и интерпретация

Результаты на задаче California Housing Regression сведены в таблицу 1. Лучшей оказалась конфигурация «расширенный глубокий режим», для которой получены $test_rmse = 0.795757$, $test_mae = 0.593770$ и $test_r2 = 0.507159$. Для конфигурации «плоская базовая конфигурация» соответствующие значения составили $test_rmse = 0.890950$, $test_mae =$

0.678868 и $\text{test_r2} = 0.382192$. Следовательно, в регрессионной постановке глубокий нечеткий контур формирования признаков дает измеримое улучшение качества по сравнению с плоской базовой моделью.

Результаты на задаче California Value Binary Geo приведены в таблице 2. Здесь лучшей оказалась конфигурация «плоская интерпретируемая конфигурация», для которой получены $\text{test_accuracy} = 0.746341$, $\text{test_precision} = 0.743073$, $\text{test_recall} = 0.735661$ и $\text{test_f1} = 0.739348$. Этот результат особенно важен для общей логики статьи, поскольку показывает, что RuFLEX не навязывает единственную оптимальную архитектуру. Для одной задачи выигрыш дает более глубокое представление, а для другой оказывается предпочтительной компактная и более прозрачная конфигурация.

Содержательная ценность платформы определяется не только итоговыми метриками. Для лучших конфигураций автоматически формируются панели интерпретации, графики функций принадлежности, история обучения, представления наиболее активных правил, скрытых нечетких концептов и траекторий их формирования. Тем самым статья получает не только сводку численных результатов, но и структурный материал, позволяющий проследить, каким образом итоговый ответ модели связан с активированными правилами, степенями принадлежности и промежуточными концептами.

6. Заключение

В статье представлена платформа RuFLEX для гибридного глубокого нечеткого обучения, ориентированная на обработку геоданных и объединяющая дифференцируемые нечеткие слои, модули глубокого нечеткого формирования признаков, расширяемый пространственный контур глубоких нечетких сверток и механизмы принятия решений на основе правил. Показано, что платформа позволяет совместно обучать параметры функций принадлежности, базы правил и внутренних представлений признаков, сохраняя интерпретируемый вывод на уровне факторов, правил и скрытых концептов.

Экспериментальная проверка на задачах регрессии и бинарной классификации подтверждает, что RuFLEX следует рассматривать именно как платформу выбора и сравнения модельных режимов. Перспективы дальнейшего развития связаны с усилением пространственного контура, расширением логической прослеживаемости правил и развитием

контрфактуальной интерпретации без изменения базовой архитектурной схемы.

9. References

1. Jang J.-S. R. ANFIS: Adaptive-Network-Based Fuzzy Inference System // IEEE Transactions on Systems, Man, and Cybernetics. 1993. Vol. 23, No. 3. P. 665-685.
2. Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems. 2019. Vol. 32.
3. Pedregosa F., Varoquaux G., Gramfort A. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. 2011. Vol. 12. P. 2825-2830.
4. Pace R. K., Barry R. Sparse Spatial Autoregressions // Statistics & Probability Letters. 1997. Vol. 33, No. 3. P. 291-297. DOI: 10.1016/S0167-7152(96)00140-X.
5. Rudin C. Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead // Nature Machine Intelligence. 2019. Vol. 1. P. 206-215. DOI: 10.1038/s42256-019-0048-x.
6. Ma X., Chen L., Deng Z. et al. Deep Image Feature Learning With Fuzzy Rules // IEEE Transactions on Emerging Topics in Computational Intelligence. 2024. Vol. 8. P. 724-737. DOI: 10.1109/TETCI.2023.3259447.
7. Lebedeffson. deep-neuro-fuzzy [Электронный ресурс]. URL: <https://github.com/lebedeffson/deep-neuro-fuzzy> (дата обращения: 13.04.2026).